AIN SHAMS UNIVERSITY

FACULTY OF ENGINEERING

i-CREDIT HOURS ENGINEERING PROGRAMS

TinyTelemetry Phase 1 — IoT Telemetry Protocol

Submitted to: Dr. Karim Emara .

Submitted by :

1. Abdelrahman Adel 23P0144
2. Monzer Mahmoud 23p0122
3. Omar Ahmed Abouraia 23P0100
4. Hossam Osama 23p0010
5. Zeina Reda 23P0181
6. logine Mohamed 23p0187

Contents

Mini-RFC (Sections 1–3) 3
- Introduction 3
- Protocol Architecture 3
- Message Formats 3
- Payload Structure 4
- Message Types 4

Project Proposal 5
- Assigned Scenario 5
- Motivation 5
- Proposed Protocol Approach 5
- Key Design Features 5

Protocol Flowchart 6

Implementation and Testing 7
- System Overview 7
- Implementation Approach 7
- Testing and Evaluation 7
- Results and Discussion 8
- Conclusion 8

Mini-RFC (Sections 1–3)

Introduction

TinyTelemetry v1 is a compact application-layer telemetry protocol that operates over UDP, enabling IoT sensors to transmit periodic environmental readings to a central collector. The protocol is designed to handle packet loss gracefully while maintaining minimal message overhead and efficient encoding.

Assumptions and Constraints

• Transport Layer: UDP (no retransmission)
• Maximum Packet Size: 200 bytes (header + payload)
• Maximum Readings per Packet: 31 (with per-reading checksum)
• Endianness: Network byte order (big-endian)
• Loss-tolerant and stateless communication model

Protocol Architecture

The protocol follows a sessionless architecture composed of Sensor Nodes (clients) and a Collector (server). Each sensor periodically sends independent packets to the collector, which logs, validates, and analyzes them. The server remains stateless and detects packet loss or duplication through sequence tracking.

Entities

Entity	Role	Description
Sensor Node (Client)	Data Source	Generates and sends sensor readings periodically using UDP.
Collector (Server)	Data Sink	Receives, validates, and logs readings. Detects duplicates and loss.

Message Formats

TinyTelemetry packets consist of a 12-byte fixed header followed by a variable payload.
The payload contains one or more sensor readings. Each reading has either 5 or 6 bytes depending on whether the checksum flag is enabled.

Header Format (12 bytes total)

Byte Offset	Field	Size (bytes)	Description
0	Ver+MsgType	1	Upper 4 bits = Version; Lower 4 bits = Message Type.
1	Flags	1	bit0=Batching, bit1=Urgent, bit2=Checksum, bit3=Encrypted, bits4–7=Reserved.

Byte Offset	Field	Size (bytes)	Description
2–3	DeviceID	2	Unique sensor identifier (0–65535).
4–7	SeqNum	4	Sequence number, increments with each packet.
8–11	Timestamp	4	Milliseconds since epoch (32-bit).

Payload Structure

Each reading has 1 byte for SensorType+UnitType, 4 bytes for the float value, and optionally 1 byte for CRC8 checksum
(if the checksum flag is enabled). A 1-byte batch count is included before the readings when batching is active.

Case	Payload per Reading	Contents
Checksum Flag = 0	5 bytes	1B Sensor+Unit, 4B Float Value
Checksum Flag = 1	6 bytes	1B Sensor+Unit, 4B Float Value, 1B CRC8

Message Types

Value	Message Type	Description
1	INIT	Sent when device starts to announce itself.
2	DATA	Carries one or more sensor readings.
3	HEARTBEAT	Indicates device is active with no new data.

TinyTelemetry's 12-byte header combined with flexible payload allows high efficiency and future extensibility.
The protocol ensures minimal overhead and low latency suitable for real-time IoT telemetry applications.

Project Proposal

TinyTelemetry v1 is a lightweight IoT telemetry protocol designed for sensor nodes that periodically send
environmental data to a central collector over UDP. The protocol focuses on efficiency, small packet size,
and reliability without retransmission. It is developed to operate within constrained devices and networks
where TCP-based protocols are unsuitable.

Assigned Scenario

The system scenario involves multiple IoT sensors that send temperature, humidity, and voltage readings
to a central server using UDP. Each sensor operates independently and transmits small, fixed-size packets
at regular intervals. The collector receives, verifies, and logs the readings in real time.

Motivation

Existing IoT protocols such as MQTT or CoAP introduce connection and header overhead, making them inefficient
for frequent, small data transmissions. TinyTelemetry minimizes this overhead by using UDP, with added support
for sequencing, batching, and optional checksum verification. The design allows simple implementation,
loss tolerance, and compact message representation suitable for low-power IoT environments.

Proposed Protocol Approach

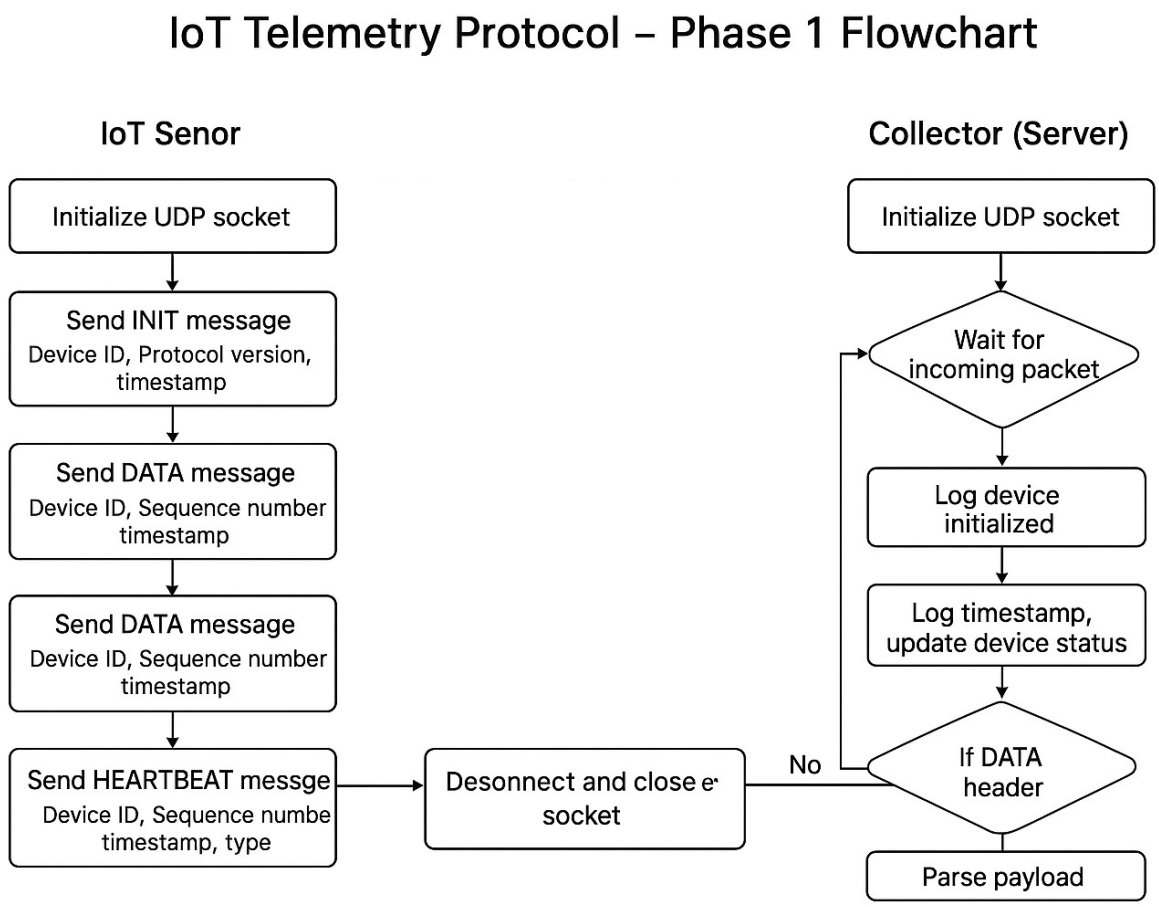
TinyTelemetry v1 is a connectionless protocol built over UDP, ensuring low-latency data delivery.
It employs a compact 12-byte header followed by a variable payload that supports single or multiple sensor readings.
The protocol incorporates versioning, flexible flags (batching, urgent, checksum, encrypted), and sequence numbers
for packet tracking. This structure provides extendibility while maintaining a strict 200-byte maximum packet size.

Key Design Features

- UDP-based low-latency operation.
- Compact 12-byte header with control flags.
- Optional checksum for per-reading data integrity.

• Support for batching multiple readings per packet (up to 31 readings).
• Reliable loss and duplication detection using sequence numbers.
• Flexible flag structure allowing protocol extensions without redesign.

Protocol Flowchart



Implementation and Testing

This section describes the implementation and testing of the TinyTelemetry v1 protocol, which demonstrates
the functional behavior of a lightweight IoT telemetry communication system operating over UDP. The implementation
comprises three main components: the Collector (server), the Sensor Node (client), and the UDP Tester tool.
Each module was designed and implemented in Python to simulate protocol functionality under realistic IoT network conditions.

System Overview

The implementation follows a client–server architecture over UDP sockets. The Collector acts as a passive data sink,
while multiple Sensor Nodes (clients) transmit telemetry readings periodically. The UDP Tester tool enables network
behavior simulation, including packet loss, duplication, and reordering, to evaluate protocol robustness.

Implementation Approach

The modules were implemented using Python's built-in socket library for UDP communication.
Packet encoding and decoding were performed using the struct library for byte-level data management.
The protocol design enforces strict byte alignment and endianness consistency to ensure interoperability.

Each function in the codebase corresponds to a specific protocol responsibility, reflecting a clean modular design.
Logging and print statements were integrated for debugging, validation, and output visualization.

Testing and Evaluation

Testing was performed by running multiple client instances concurrently, each sending telemetry data to a single Collector.
The following aspects were evaluated:

- Packet integrity under simulated loss and duplication.
- Sequence tracking and reordering detection.
- Batching and checksum correctness.
- Throughput and packet size compliance (≤ 200 bytes).

Observations confirmed that the protocol achieved efficient low-latency communication while maintaining acceptable loss tolerance.

Losses did not result in retransmissions, as expected in a UDP-based design, but were detectable through sequence discontinuities.
Checksum validation ensured reading-level integrity when enabled.

Results and Discussion

The TinyTelemetry v1 prototype successfully demonstrated the feasibility of lightweight IoT telemetry communication
with minimal overhead. The system operated effectively within the defined 200-byte packet constraint, enabling fast and
compact data exchange suitable for embedded and low-power devices.

The modular architecture allows future extensions, including encryption or adaptive retransmission mechanisms,
without altering the base protocol structure.

Conclusion

This implementation verified that TinyTelemetry v1 achieves its design goals: efficiency, reliability without retransmission,
and low communication overhead. The use of UDP proved effective for constrained environments where simplicity and
real-time operation are prioritized.